

Agregando un Filtro a la Etapa 2-BD

Objetivo: agregarle a `resultado.php` un buscador que permita filtrar las personas por nombre.

Punto de partida: tenés la aplicación **Etapa 2-BD** funcionando — formulario que guarda en MySQL y una página `resultado.php` que muestra **todas** las personas con `obtenerTodasLasPersonas()`.

Al terminar, vas a poder escribir "Jul" en una cajita, apretar **Buscar** y ver solo las personas cuyo nombre contiene "Jul" (Julián, Julia, etc.). Si dejás la cajita vacía y apretás Buscar, ves a todos de nuevo.

0. Idea general: ¿qué vamos a tocar?

Un filtro tiene **dos partes** que trabajan juntas, como un equipo:

1. **Backend (la base de datos):** una función nueva que sabe hacer `SELECT ... WHERE nombre LIKE '%algo%'`.
2. **Frontend + conexión (la página visible):** leer lo que el usuario escribió, llamar a esa función y mostrar el resultado.

Solo vamos a modificar **2 archivos**:

Archivo	Qué cambiamos
<code>bd/gestionBaseDatos.php</code>	Agregamos una función nueva: <code>obtenerPersonasConFiltro()</code>
<code>resultado.php</code>	3 cambios chiquitos: (A) leer el texto buscado, (B) agregar la cajita de búsqueda, (C) mejorar el cartel de "no hay resultados"

`index.php` y `procesando.php` **no se tocan**. El filtro es solo para *mostrar* datos, no para guardarlos.

1. Paso 1 — El backend: crear `obtenerPersonasConFiltro()` en `bd/gestionBaseDatos.php`

1.1 ¿Qué hace esta función?

En la Etapa 2 tenías:

```
function obtenerTodasLasPersonas($conexion) {  
    $sql = "SELECT * FROM personas ORDER BY id DESC";  
    $stmt = $conexion->prepare($sql);  
    $stmt->execute();  
    return $stmt->fetchAll(PDO::FETCH_ASSOC);  
}
```

Siempre trae **todas** las filas. Ahora queremos algo más inteligente:

Si el usuario no buscó nada → trae a todos.

Si buscó "Jul" → trae solo los que tienen "Jul" en el nombre.

En SQL eso se hace con **LIKE** y el comodín **%**:

```
-- Trae todos los que tengan "jul" en cualquier parte del nombre  
SELECT * FROM personas WHERE nombre LIKE '%jul%'  
-- '%jul%' matchea "Julián", "Julia", "Julieta"
```

% significa "cualquier texto, de cualquier largo". **'Jul%'** sería "empieza con Jul", **'%Jul'** sería "termina con Jul".

1.2 La función nueva

Abrí `bd/gestionBaseDatos.php` y al final del archivo, antes de `eliminarPersona()`, agregó esta función (captura 4-`obtenerPersonaConFiltro.png`):

```

119  /**
120  * Obtiene las personas que cumplan con el filtro de búsqueda.
121  * Acepta una búsqueda opcional por NOMBRE de la persona.
122  * LIKE: se usa para buscar textos que CONTENGAN el termino buscado.
123  * '%' es un comodín: '%jul%' matchea "Julian", "Julia", etc.
124  *
125  * @param PDO $conexion Conexión a la base de datos
126  * @param string $busqueda Término de búsqueda (filtra por nombre de persona)
127  * @return array Array de personas con datos de la ciudad incluidos
128  */
129  function obtenerPersonasConFiltro($conexion, $busqueda = '') {
130      // ORDER BY id DESC: ordena de mayor a menor (los mas nuevos primero)
131      $sql = "SELECT * FROM personas p";
132
133      // Si hay término de búsqueda, agregamos un WHERE con LIKE
134      $parametros = [];
135      if ($busqueda !== '') {
136          $sql .= " WHERE p.nombre LIKE ?";
137          $parametros[] = '%' . $busqueda . '%';
138      }
139
140      $sql .= " ORDER BY p.id DESC";
141
142      $stmt = $conexion->prepare($sql);
143      $stmt->execute($parametros);
144
145      //fetchAll(PDO::FETCH_ASSOC) retorna todos los registros como array asociativo
146      return $stmt->fetchAll(PDO::FETCH_ASSOC);
147  }

```

```

/**
 * Obtiene las personas que cumplan con el filtro de búsqueda.
 * @param PDO $conexion Conexión a la base de datos
 * @param string $busqueda Término de búsqueda (filtra por nombre)
 * @return array Array de personas
 */
function obtenerPersonasConFiltro($conexion, $busqueda = '') {
    // Consulta base
    $sql = "SELECT * FROM personas p";

    // Si hay término de búsqueda, agregamos un WHERE con LIKE
    $parametros = [];
    if ($busqueda !== '') {
        $sql .= " WHERE p.nombre LIKE ?";
        $parametros[] = '%' . $busqueda . '%';
    }

    $sql .= " ORDER BY p.id DESC";

    $stmt = $conexion->prepare($sql);
    $stmt->execute($parametros);

    return $stmt->fetchAll(PDO::FETCH_ASSOC);
}

```

Puntos clave para entender el código:

Línea	Qué significa
<code>\$busqueda = ''</code>	El parámetro es opcional . Si nadie lo pasa, vale <code>''</code> (vacío) y trae a todos.
<code>\$sql = "SELECT * FROM personas p"</code>	<code>p</code> es un alias (apodo) para la tabla <code>personas</code> . Después podemos escribir <code>p.nombre</code> en vez de <code>personas.nombre</code> .
<code>if (\$busqueda !== '')</code>	Solo agregamos el WHERE si realmente hay algo que buscar.
<code>\$sql .= " WHERE p.nombre LIKE ?"</code>	El <code>.=</code> concatena : le agrega texto a la consulta que ya teníamos.
<code>\$parametros[] = '%' . \$busqueda . '%'</code>	Armamos <code>'%Jul%'</code> . Va entre <code>%</code> para que busque "que contenga".
<code>? + execute(\$parametros)</code>	Prepared statement : el <code>?</code> se reemplaza de forma segura. Nunca concatenes <code>\$_GET</code> directo en el SQL (evitás inyección SQL). Si <code>\$busqueda</code> está vacío, <code>\$parametros</code> queda como <code>[]</code> y <code>execute([])</code> simplemente trae todos.

No borres `obtenerTodasLasPersonas()`. Dejala como está; la nueva función la complementa. Te sirve para comparar o por si otro archivo la usa.

2. Paso 2 — Conectar el filtro en `resultado.php` (parte PHP, arriba de todo)

En `resultado.php`, bien arriba donde dice `<?php ... ?>`, hacés el cambio que se ve en la captura `1-obtenerPersonas.png`:

```

8
9 // Incluimos el archivo con la conexion y funciones de la BD
10 require_once 'bd/gestionBaseDatos.php';
11
12 // Obtenemos la conexion a MySQL
13 $conexion = obtenerConexion();
14
15 // Consultamos TODAS las personas guardadas en la base de datos
16 // La funcion nos devuelve un array con cada persona como un array asociativo
17 // $personas = obtenerTodasLasPersonas($conexion);
18
19 // Recibimos el termino de busqueda desde la URL (si existe)
20 $busqueda = isset($_GET['busqueda']) ? trim($_GET['busqueda']) : '';
21
22 // La funcion recibe el termino: si esta vacio, trae todas las personas;
23 // si no, filtra por nombre usando LIKE
24 $personas = obtenerPersonasConFiltro($conexion, $busqueda);
25
26 ?>

```

Antes (Etapa 2 sin filtro):

```

require_once 'bd/gestionBaseDatos.php';
$conexion = obtenerConexion();
$personas = obtenerTodasLasPersonas($conexion);

```

Después (con filtro):

```

require_once 'bd/gestionBaseDatos.php';
$conexion = obtenerConexion();

// Esta línea vieja la comentamos o la borramos:
// $personas = obtenerTodasLasPersonas($conexion);

// Recibimos el término de búsqueda desde la URL (si existe)
$busqueda = isset($_GET['busqueda']) ? trim($_GET['busqueda']) : '';

// La función recibe el término: si está vacío, trae todas las personas;
// si no, filtra por nombre usando LIKE
$personas = obtenerPersonasConFiltro($conexion, $busqueda);

```

¿Qué estás haciendo línea por línea?

1. `$_GET['busqueda']` — Cuando el usuario apreta "Buscar", el navegador va a `resultado.php?busqueda=Jul`. Todo lo que va después de `?` viaja por **GET** y PHP lo guarda en el array `$_GET`.

2. `isset($_GET['busqueda'])` — Pregunta: "¿vino algo llamado `busqueda` en la URL?". La primera vez que abrís la página, no vino nada, entonces da `false`.
3. `? trim(...)` : `'` — Es un **operador ternario** (un `if` en una línea). Si vino algo, lo limpia con `trim()` (saca espacios extra: `" Jul "` → `"Jul"`). Si no vino nada, usa `'` (vacío).
4. `$personas = obtenerPersonasConFiltro(...)` — Llamás a la función nueva. Si `$busqueda` es `'`, la función hace un `SELECT` sin `WHERE` y trae a todos. Si es `"Jul"`, hace `WHERE nombre LIKE '%Jul%'`.

¿Por qué GET y no POST? Porque un filtro es una **consulta**, no un guardado. Con GET el término queda en la URL (`?busqueda=Jul`) y podés compartir el link, recargar la página o volver atrás y el filtro sigue ahí. POST es para cuando cambiás datos (guardar/eliminar), y por eso `index.php` usa POST.

3. Paso 3 — El buscador visible (formulario HTML en `resultado.php`)

Ahora que la parte de arriba ya sabe filtrar, falta la **cajita** donde el usuario escribe.

Dentro de `<div class="card-body">`, justo antes del `<?php if (empty($personas)): ?>`, agregá el formulario que se ve en la captura 2-
`formularioFiltro.png`:

```

<div class="card-body">

<!--
    BUSCADOR:
    - method="GET": el termino viaja en la URL (?busqueda=...)
    - action="resultado.php": recarga esta misma pagina con el filtro
    - value=... : deja el termino escrito despues de buscar
-->
    <form method="GET" action="resultado.php" class="row g-2 mb-3">
        <div class="col-md-9">
            <input type="text" name="busqueda" class="form-control"
                placeholder="Buscar por nombre de persona..."
                value="<?php echo htmlspecialchars($busqueda); ?>"
            </div>
            <div class="col-md-3">
                <button type="submit" class="btn btn-success w-100">Buscar</button>
            </div>
        </form>

    <?php if (empty($personas)): ?>

    <!-- Si no hay personas en la BD, mostramos un mensaje informativo -->

```

```

<!--
    BUSCADOR:
    - method="GET": el término viaja en la URL (?busqueda=...)
    - action="resultado.php": recarga esta misma página con el filtro
    - value="...": deja el término escrito después de buscar
-->
<form method="GET" action="resultado.php" class="row g-2 mb-3">
    <div class="col-md-9">
        <input type="text" name="busqueda" class="form-control"
            placeholder="Buscar por nombre de persona..."
            value="<?php echo htmlspecialchars($busqueda); ?>"
        </div>
        <div class="col-md-3">
            <button type="submit" class="btn btn-success w-100">Buscar</button>
        </div>
    </form>

```

Desglose para no perderte:

Atributo	Para qué es
<code>method="GET"</code>	Le dice al navegador: "poné los datos en la URL". Al enviar, la URL queda <code>resultado.php?busqueda=lo-que-escribi</code> .

Atributo	Para qué es
<code>action="resultado.php"</code>	"Cuando apretes Buscar, recargá esta misma página". Es la misma página la que se filtra a sí misma.
<code>name="busqueda"</code>	El nombre de la clave que llegará a <code>\$_GET</code> . Tiene que coincidir con <code>\$_GET['busqueda']</code> del Paso 2.
<code>placeholder="Buscar por..."</code>	Texto gris que se ve cuando la cajita está vacía. Es solo ayuda visual.
<code>value="<?php echo htmlspecialchars(\$busqueda); ?>"</code>	¡Muy importante! Deja escrito lo que buscaste después de apretar Buscar. Sin esto, la cajita quedaría vacía y no sabrías qué filtraste. <code>htmlspecialchars()</code> evita que un texto con <code><</code> o <code>"</code> rompa el HTML.
<code>class="row g-2 mb-3" / col-md-9 / col-md-3</code>	Clases de Bootstrap para que la cajita ocupe 9 columnas y el botón 3, uno al lado del otro.

4. Paso 4 — Mejorar el cartel de "no hay resultados"

En la Etapa 2 el cartel era siempre:

```
<div class="alert alert-info">No hay personas registradas aún.</div>
```

Ahora hay **dos situaciones distintas** y queremos mensajes distintos:

1. La tabla está vacía de verdad → "No hay personas registradas aún."
2. La tabla tiene gente, pero nadie matchea tu búsqueda → "No se encontraron personas con ese nombre."

Reemplazá ese bloque por lo que se ve en la captura `3-ifEmpty.png`:

```

<?php if (empty($personas)):?>
    <!-- Si no hay personas en la BD, mostramos un mensaje informativo -->
    <!-- <div class="alert alert-info">No hay personas registradas aun.</div> -->
    <div class="alert alert-info">
        <?php echo ($busqueda !== '') ? 'No se encontraron personas con ese nombre.' : 'No hay personas registradas aun.'; ?>
    </div>

<?php else: ?>
<div class="table-responsive">
    <table class="table table-striped table-hover align-middle">
        <!-- table-striped: filas con colores alternados -->
        <!-- table-hover: resalta la fila cuando pasas el mouse -->
        <thead class="table-dark">
            <tr>
                <th>#</th>
                <th>Nombre</th>
                <th>Apellido</th>

```

```

<?php if (empty($personas)): ?>
    <!-- Si no hay personas en la BD, mostramos un mensaje informativo -->
    <!-- <div class="alert alert-info">No hay personas registradas aun.
</div> -->
    <div class="alert alert-info">
        <?php echo ($busqueda !== '') ? 'No se encontraron personas con ese
nombre.' : 'No hay personas registradas aun.'; ?>
    </div>
<?php else: ?>

```

¿Cómo funciona?

- `empty($personas)` — ¿el array vino vacío? Puede ser por las dos razones de arriba.
- `($busqueda !== '') ? ... : ...` — Otro ternario: si `$busqueda` **no** está vacío, es porque el usuario buscó algo y no hubo coincidencias → mostrará el primer mensaje. Si está vacío, es porque la tabla realmente no tiene filas → mostrará el segundo.

El resto del archivo (el `foreach` que dibuja la tabla, el botón Eliminar, el `Volver al formulario`) queda exactamente igual.

5. Paso 5 — Probar que todo funciona

1. Levantá el servidor desde la carpeta correcta:

```

cd etapa2-BD-Filtro
php -S localhost:8000

```

2. Abrí <http://localhost:8000/resultado.php>. Deberías ver el buscador arriba de la tabla, como en la captura [5-frontendFiltro.png](#):

Personas Registradas

2 registros

Buscar por nombre de persona...

Buscar

#	Nombre	Apellido	DNI	CUIT	Nacimiento	Email	Telefono	Ciudad	Acciones
2	Maria	Perez	31333456	20313334569	1987-05-12	julianconde.ipeayt186@gmail.com	3534111579	Las Higueras	Eliminar
1	Julián	Conde	30090561	20300905618	1983-04-10	jconde@ac.unrc.edu.ar	502	RIO CUARTO	Eliminar

Volver al formulario

3. Probá estos casos:

Qué hacés	Qué esperás ver en la URL	Qué ves en la tabla
Entrás sin buscar	resultado.php	Todas las personas
Escribís Jul y apretás Buscar	resultado.php?busqueda=Jul	Solo Julián, Julia, etc.
Escribís ZZZ (que no existe)	resultado.php?busqueda=ZZZ	Cartel: "No se encontraron personas con ese nombre."
Borrar el texto y apretar Buscar de nuevo	resultado.php?busqueda=	Vuelven todos

Si cambiás la URL a mano ([?busqueda=ma](#)) y apretás Enter, también filtra. Eso es la magia de GET: el filtro es parte del link.

4. Revisá que **Eliminar** siga funcionando (usa POST y va a [procesando.php](#), no toca el filtro).

6. Resumen: cómo viaja un filtro de punta a punta

```

1. Usuario escribe "Jul" y aprieta Buscar
    |
    ▼ GET (el navegador arma la URL)
    resultado.php?busqueda=Jul
    |
    ▼ PHP lee $_GET
    $busqueda = "Jul"
    |
    ▼ Llama a la función con el término
    obtenerPersonasConFiltro($conexion, "Jul")
    |
    ▼ SQL armado dinámicamente
    SELECT * FROM personas p WHERE p.nombre LIKE '%Jul%' ORDER BY p.id DESC
    |
    ▼ MySQL devuelve solo los que matchean
    $personas = [ Julián, Julia ]
    |
    ▼ foreach dibuja las filas
    <table> ... solo 2 filas ...
    |
    ▼ value="Jul" deja la palabra escrita
    [ Buscar por nombre... Jul ] [Buscar]
  
```

Todo el trabajo extra fueron ~**20 líneas**: una función con **LIKE** y un formulario con **GET**.

7. Para seguir jugando (desafíos)

Una vez que este filtro anda, podés animarte a:

- **Filtrar por apellido también:** cambiá el **WHERE** a **WHERE p.nombre LIKE ? OR p.apellido LIKE ?** y pasá dos parámetros iguales (`['%'.$busqueda.'%', '%'.$busqueda.'%']`).
- **Filtrar por ciudad:** **WHERE p.ciudad LIKE ?**.
- **Búsqueda insensible a mayúsculas:** MySQL con **utf8mb4** ya lo hace, pero probá buscar **julian** en minúsculas y ver si encuentra **Julián**.
- **Botón "Limpiar filtro":** un link a **resultado.php** sin parámetros (`<a href="resultado.php" class="btn btn-secondary">Limpiar</a>`) que vuelve a mostrar todo.

¡Con esto ya tenés tu primera funcionalidad que interactúa con la base de datos para *consultar* de forma inteligente!